Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Taller de diseño digital (EL3313)

Prof. Kaleb Alfaro Bonilla, M.Sc.

Estudiantes:

2021155576 - Bolaños Solís Oscar

2022207101 - Bou Espinoza Raquel

2022195304 - Lobo Campos Andrés

2021021770 - Sánchez Herrera Angélica

Lab. 1: Documentación

I Semestre 2026

Resumen

En el presente documento se desarrollan cuatro ejercicios prácticos enfocados en la introducción al diseño digital mediante lenguajes de descripción de hardware (HDL) y el uso de herramientas EDA para síntesis y simulación. Se emplean los lenguajes SystemVerilog, para simulación, y Verilog, para la implementación en hardware. Se utiliza la FPGA Nexys 4 DDR en los ejercicios 3.1, 3.3 y 3.4, los cuales abordan respectivamente el control de switches y LEDs, la modulación por ancho de pulso (PWM) y la comunicación serial asíncrona. Asimismo, se realiza la verificación del comportamiento de cada diseño mediante simulaciones.

Tabla de contenidos

Resumen 2
Ejercicios prácticos 5
Ejercicio 3.1- Switches, botones y LEDs 5
Diseño: 5
Código 6
Constrains 6
Designed Sources 7
Simulation Sources 7
Resultados de simulación: 7
Verificación de funcionamiento en la FPGA 8
Ejercicio 3.2- Multiplexor 4-to-1 8
Diseño: 8
Código 9
Designed Sources y Simulation Sources 9
Resultados de simulación 10
Ejercicio 3.3- Modulación de ancho de pulso 12
Diseño: 12
Código 13
Constrains 13
Designed Sources 13
Simulation Sources 13
Resultados de simulación: 14
Verificación de funcionamiento en la FPGA 14
Ejercicio 3.4- Interfaz Serial Asíncrona 14
Diseño: 16
Código: 17
Constrains y Designed Sources 17
Simulation Sources 18

Resultados de simulación: 18
Verificación de funcionamiento en la FPGA 18
Referencias 19

Ejercicios prácticos

Ejercicio 3.1- Switches, botones y LEDs

1. Diseñe un módulo que reciba como entradas los 4 interruptores hacia los pines de la FPGA que se utilizará en este laboratorio.
2. Las salidas del módulo se mostrarán en los 4 LEDs disponibles en la tarjeta de desarrollo.
3. El bloque a desarrollar debe convertir el código ingresado por los interruptores por el correspondiente a su complemento a 2 hacia los LEDs.
4. Implemente un banco de pruebas (testbench) para validar el funcionamiento de su diseño.
5. Realice una validación de su diseño en la tarjeta con FPGA.

Diseño:

Preliminarmente, se realizó el préstamo de la FPGA Nexys 4 DDR Artix-7. Se agregó la board y el archivo .xdc a Vivado.

Primeramente, se analizó el funcionamiento y lógica del ejercicio, de manera que se evaluó el comportamiento de los switches que estén en alto y bajo representen una señal de 4 bits, utilizando los switches R15, M13, L16 y J15 de la FPGA en orden MSB respectivamente, seguidamente la señal es invertida por lo que los valores en alto cambian a bajos y los valores que estaban en bajo cambian a alto, a este nuevo valor se le suma uno y representa el complemento a 2, este debe ser visualizado mediante los leds N14, J13, K15 y H17 de la FPGA se deberá de comportar respecto a los valores que se muestran en la tabla.

Tabla 1. Comportamiento de entrada y salidas esperadas

Clasificación	**Entradas**	**Salidas**
	Número binario $SW_3\,SW_2\,SW_1\,SW_0$	**Complemento a 2** $LED_3\,LED_2\,LED_1\,LED_0$
A	0001	1111
B	0010	1110
C	0011	1101
D	0100	1100
E	0101	1011
F	0110	1010
G	0111	1001
H	1000	1000
I	1001	0111
J	1010	0110
K	1011	0101
L	1100	100
M	1101	0100
N	1110	0011
O	1111	0001

Código

En el siguiente enlace se encuentra el repositorio del Ejercicio 3.1

https://github.com/Oscar-Electronics/Taller_diseño_digital/commit/5d4078d02b0bf19f370f1f54aac4c500966956e1

Constrains

Se creo un archivo llamado MasterFile en la que se dejaron únicamente las constrains necesarias de los switches y leds para la implementación, las cuales se tomaron del repositorio digilent-xdc de [1].

https://github.com/Oscar-Electronics/Taller_diseño_digital/blob/main/Laboratorio%201/problema%203.1/Master_File.xdc

Designed Sources

Se creo el módulo ejercicio_3_1, en el que se diseñó el complemento a 2 para la FPGA.
En el siguiente enlace se puede acceder directamente al módulo en el repositorio de GitHub:

https://github.com/Oscar-Electronics/Taller_diseño_digital/blob/main/Laboratorio%201/problema%203.1/ejercicio_3_1.v

Simulation Sources

Se creo el módulo tb_ejercicio_1, en el que se simula y se espera su respectivo complemento a 2 de A, B, C, D, E y H que se muestran en la tabla 1.

En el siguiente enlace se puede acceder directamente al testbench en el repositorio GitHub:

https://github.com/Oscar-Electronics/Taller_diseño_digital/blob/main/Laboratorio%201/problema%203.1/tb_ejercicio_1.sv

Resultados de simulación:

Una vez realizado el run behavioral simulation se obtiene el siguiente oscilograma donde se comprueba el comportamiento de la FPGA bajo la simulación implementada y se comporta tal cual fue esperado respecto a la tabla 1.

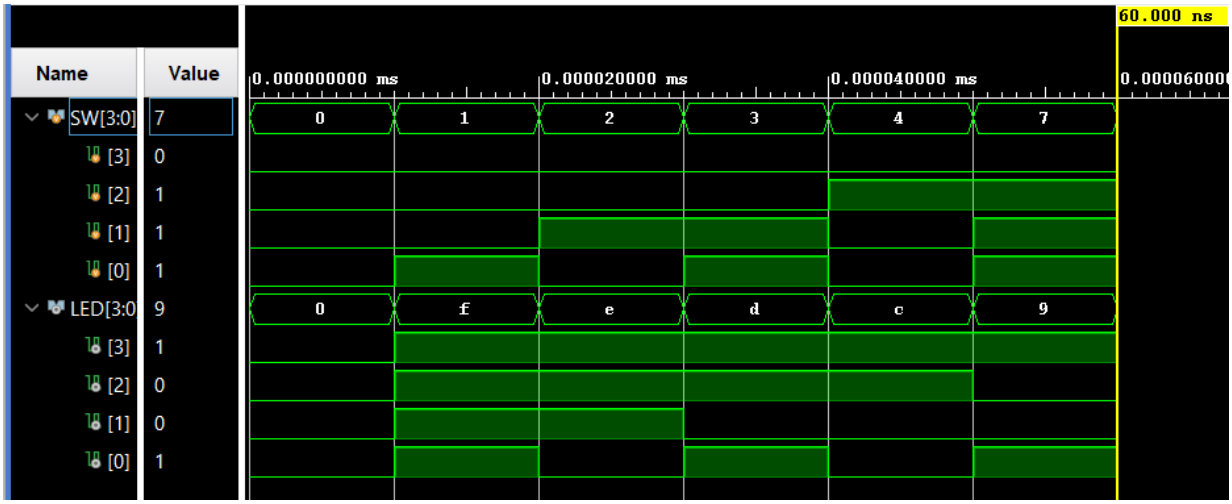

Fig. 1 Oscilograma del comportamiento esperado de los switches con los leds

Verificación de funcionamiento en la FPGA

Una vez obtenido el resultado esperado en el osciloscopio se procede a generar el bitstream y la implementación en la FPGA.

En el siguiente enlace se puede verificar la funcionalidad de la implementación realizada:

https://youtu.be/MkBTeuorBac

Ejercicio 3.2- Multiplexor 4-to-1

1. Diseñe un multiplexor de cuatro entradas, parametrizado, para un ancho de datos (bus width) de entrada y salida variable.
2. Realice un banco de prueba (testbench) en el que se muestre de manera simple el funcionamiento del multiplexor. Muestre el resultado de la prueba para anchos de datos de 4, 8 y 16 bit, para las cuatro selecciones de entrada y para al menos un conjunto de datos de entrada de 50 datos en cada caso.
3. Ejecute su prueba y asegúrese de que todos los errores y las advertencias producidas por la simulación hayan sido debidamente atendidas y corregidas.

Diseño:

Se hizo la tabla de verdad de un mux 4-1, para comprender la lógica y funcionamiento del problema

Tabla 2. Comportamiento de un mux 4-1

Selección 1 (S_1)	Selección 0 (S_0)	Salida (Y)
0	0	Input 0
0	1	Input 1
1	0	Input 2
1	1	Input 3

Posteriormente se decidió hacer una máquina de estados, para comprender mejor como se vería las transiciones de salidas según las selecciones, para implementarlo en un módulo:

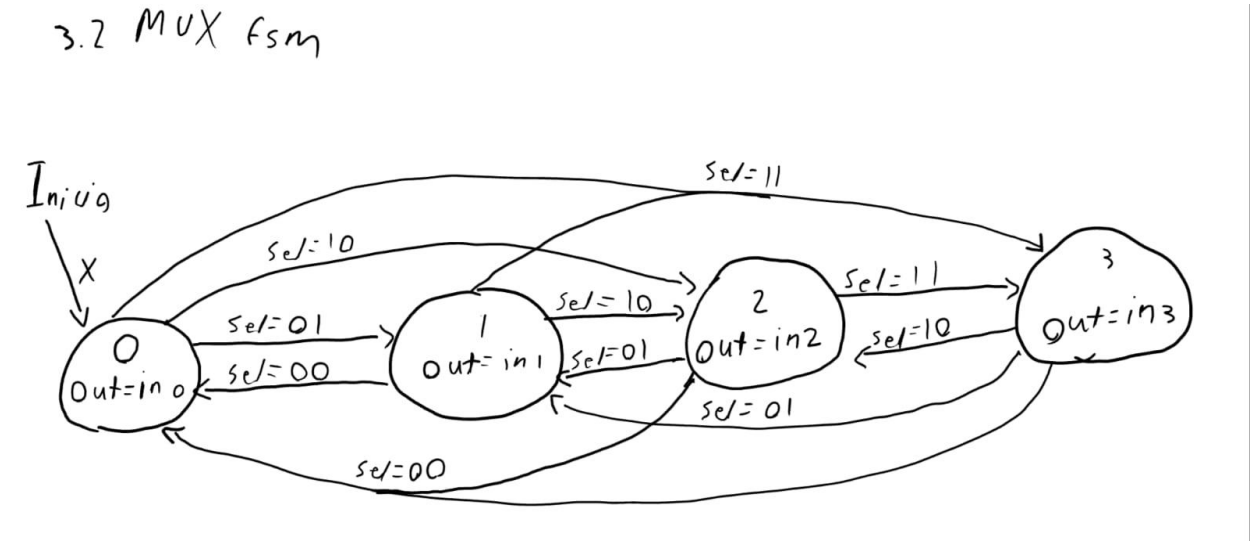

Fig. 2 Maquina de estado Mux

Hay que tomar en cuenta que esta salidas, el ancho de banda cambia ya que se ocupa un mux de 4, 8 y 16 de ancho de banda, esto lo que significa es que la cantidad de bits que componen a Input 0,1,2 y 3 va a cambiar según el bus width que se use, por eso en el código se dejó el bus width como parámetro ya que las señales internas de input el tamaño se decide según este parámetro, además para hacer los 50 datos de entrada decidimos agregar una valor random a las salidas que cambian cada 4 ciclos, esto se hizo con $random y el residuo de dividir el numero de la prueba (empieza desde 0 hasta 50) entre 4 .

Código

Designed Sources y Simulation Sources

https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/39409e7e5de8d142ae922d6f70859434e66ab2de/Laboratorio%201/problema%203.2

Además que el código de simulación (testbench) crea un archivo llamado resultados_mux_3_texto, para una mejor visualización de los valores esperados y el recibido.

Resultados de simulación

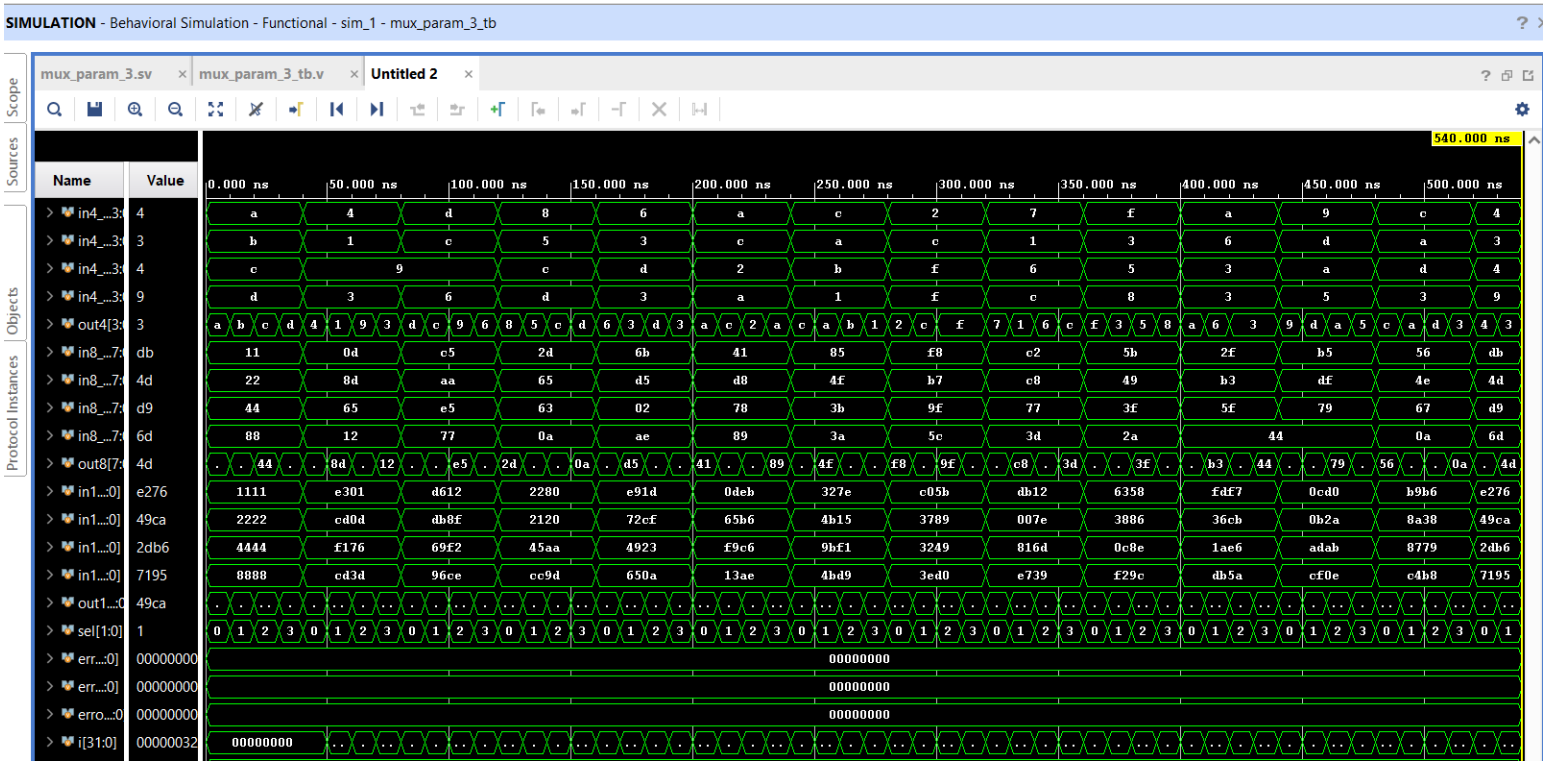

Fig. 3 Comportamiento del Mux

Como es posible ver las señales de error se mantienen en constante 0, lo que significa que no hay ningún error en cualquier de los diferentes bus width, entonces todo funciona bien. Lo anterior es más fácil de visualizar en la siguiente figura que es el archivo de texto mencionado anteriormente.

```
PRUEBA DEL MULTIPLEXOR
VALORES FIJOS:
in4_0=a, in4_1=b, in4_2=c,                                  in4_3=d
in8_0=11, in8_1=22, in8_2=44,                               in8_3=88
in16_0=1111, in16_1=2222, in16_2=4444,                      in16_3=8888

Sel     MUX4    Esperado        MUX8            Esperado        MUX16           Esperado        Resultado
0       a       a               11      11              1111    1111            OK
1       b       b               22      22              2222    2222            OK
2       c       c               44      44              4444    4444            OK
3       d       d               88      88              8888    8888            OK
PRUEBA CON 50 DATOS ALEATORIOS
Ciclo   Sel     MUX4    Esperado        MUX8            Esperado        MUX16           Esperado        Resultado
0       0       4       4               0d      0d              e301    e301            OK
1       1       1       1               8d      8d              cd0d    cd0d            OK
2       2       9       9               65      65              f176    f176            OK
3       3       3       3               12      12              cd3d    cd3d            OK
4       0       d       d               c5      c5              d612    d612            OK
5       1       c       c               aa      aa              db8f    db8f            OK
6       2       9       9               e5      e5              69f2    69f2            OK
7       3       6       6               77      77              96ce    96ce            OK
8       0       8       8               2d      2d              2280    2280            OK
9       1       5       5               65      65              2120    2120            OK
10      2       c       c               63      63              45aa    45aa            OK
11      3       d       d               0a      0a              cc9d    cc9d            OK
12      0       6       6               6b      6b              e91d    e91d            OK
13      1       3       3               d5      d5              72cf    72cf            OK
14      2       d       d               02      02              4923    4923            OK
15      3       3       3               ae      ae              650a    650a            OK
16      0       a       a               41      41              0deb    0deb            OK
17      1       c       c               d8      d8              65b6    65b6            OK
18      2       2       2               78      78              f9c6    f9c6            OK
19      3       a       a               89      89              13ae    13ae            OK
20      0       c       c               85      85              327e    327e            OK
21      1       a       a               4f      4f              4b15    4b15            OK
22      2       b       b               3b      3b              9bf1    9bf1            OK
23      3       1       1               3a      3a              4bd9    4bd9            OK
24      0       2       2               f8      f8              c05b    c05b            OK
25      1       c       c               b7      b7              3789    3789            OK
26      2       f       f               9f      9f              3249    3249            OK
27      3       f       f               5c      5c              3ed0    3ed0            OK
28      0       7       7               c2      c2              db12    db12            OK
29      1       1       1               c8      c8              007e    007e            OK
30      2       6       6               77      77              816d    816d            OK
31      3       c       c               3d      3d              e739    e739            OK
32      0       f       f               5b      5b              6358    6358            OK
33      1       3       3               49      49              3886    3886            OK
34      2       5       5               3f      3f              0c8e    0c8e            OK
35      3       8       8               2a      2a              f29c    f29c            OK
36      0       a       a               2f      2f              fdf7    fdf7            OK
37      1       6       6               b3      b3              36cb    36cb            OK
38      2       3       3               5f      5f              1ae6    1ae6            OK
39      3       3       3               44      44              db5a    db5a            OK
40      0       9       9               b5      b5              0cd0    0cd0            OK
41      1       d       d               df      df              0b2a    0b2a            OK
42      2       a       a               79      79              adab    adab            OK
43      3       5       5               44      44              cf0e    cf0e            OK
44      0       c       c               56      56              b9b6    b9b6            OK
45      1       a       a               4e      4e              8a38    8a38            OK
46      2       d       d               67      67              8779    8779            OK
47      3       3       3               0a      0a              c4b8    c4b8            OK
48      0       4       4               db      db              e276    e276            OK
49      1       3       3               4d      4d              49ca    49ca            OK
RESUMEN FINAL
Errores MUX4 (4 bits):  0
Errores MUX8 (8 bits):  0
Errores MUX16 (16 bits): 0

 TODAS LAS PRUEBAS PASARON CORRECTAMENTE
```

Fig. 4 Archivo de texto generado en la simulación del Mux.

Ejercicio 3.3- Modulación de ancho de pulso

1. Diseñe un módulo que reciba como entrada un código de 4 bits por medio de interruptores.
2. El módulo debe utilizar dicha entrada como código de nivel para una modulación PWM que
   se enviará como salida a un pin de la FPGA. El ancho del pulso debe ser alrededor de 1ms.
3. Se colocará como carga un LED de la tarjeta.
4. Valide el funcionamiento de su diseño mediante un testbench exhaustivo.
5. Realice una validación de su diseño en la tarjeta con FPGA que se utilizará a lo largo del curso.

Diseño:

Primeramente, se analizó el funcionamiento y lógica del ejercicio, de manera que se evaluó el comportamiento de los switches que estén en alto y bajo representen una señal de 4 bits, utilizando los switches R15, M13, L16 y J15 de la FPGA en orden MSB respectivamente, seguidamente el valor de los interruptores de 4 bits se utiliza para definir el ciclo de trabajo de la señal PWM, este se calcula como la fracción del período total durante la cual la señal permanece en alto. Para un período de 1 ms, el tiempo en alto se determina mediante la expresión. Esto permite obtener 16 niveles de intensidad luminosa del LED.

La modulación de ancho de pulso PWM debe ser visualizada mediante el LED16G donde la FPGA se deberá de comportar respecto a los valores que se muestran en la Tabla 3.

Tabla 3. Comportamiento de entrada y salidas esperadas para Ejercicio 3.3

	Número binario $SW_3\,SW_2\,SW_1\,SW_0$	**Decimal**	**Ciclo de trabajo**	**Brillo**
A	0000	0	0%	Apagado
B	0001	1	6.25%	Muy tenue
C	0010	2	12.5%	Tenue
D	0100	4	25%	Bajo
E	1000	8	50%	Medio
F	1100	12	75%	Alto
G	1111	15	93.75%	Casi máximo

Código

En el siguiente enlace se encuentra el repositorio del Ejercicio 3.3

https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/main/Laboratorio%201/problema%203.3

Constrains

Se creó un archivo llamado MasterFile en la que se dejaron únicamente las constrains necesarias de los switches y el led para la implementación, las cuales se tomaron del repositorio digilent-xdc de [1].

https://github.com/Oscar-Electronics/Taller_diseño_digital/blob/main/Laboratorio%201/problema%203.3/MasterFileCons.xdc

Designed Sources

Se creó el módulo Ejer_3_3.v, en el que se diseñó el PWM para la FPGA.

En el siguiente enlace se puede acceder directamente al módulo en el repositorio de GitHub:

https://github.com/Oscar-Electronics/Taller_diseño_digital/blob/main/Laboratorio%201/problema%203.3/Ejer_3_3.v

Simulation Sources

Se creo el módulo Ejer_3_3tb.sv, en el que se simula y se espera el respectivo ancho de pulso PW para A, B, C, D, E, F y G que se muestran en la Tabla 3.

En el siguiente enlace se puede acceder directamente al testbench en el repositorio GitHub:

https://github.com/Oscar-Electronics/Taller_dise-o_digital/blob/main/Laboratorio%201/problema%203.3/Ejer_3_3tb.sv

Resultados de simulación:

Una vez realizado el run behavioral simulation se obtiene el siguiente oscilograma de la Fig 4 donde se comprueba el comportamiento de la FPGA bajo la simulación implementada y se comporta tal cual fue esperado respecto a la Tabla 3.

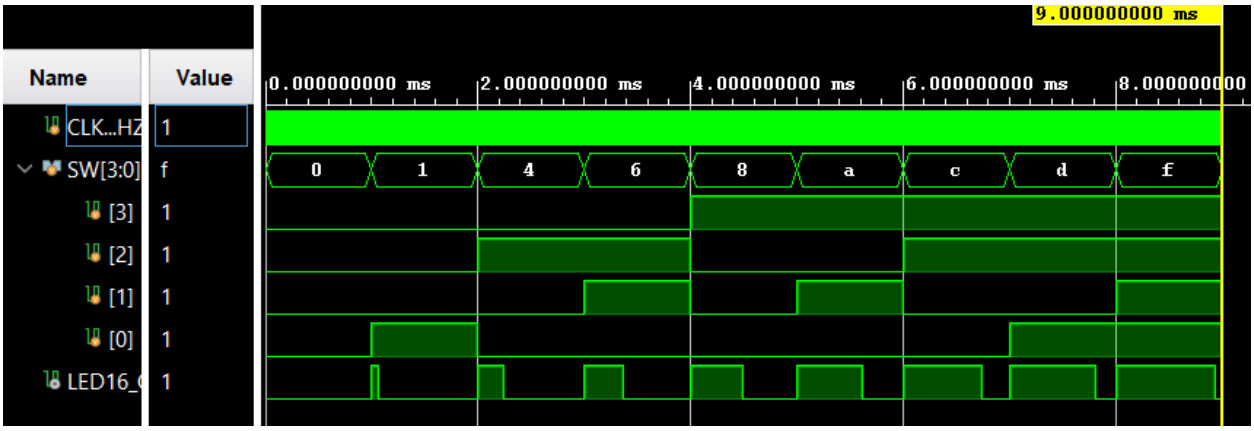

Fig 4. Oscilograma del Ejercicio 3.3

Verificación de funcionamiento en la FPGA

Una vez obtenido el resultado esperado en el osciloscopio se procede a generar el bitstream y la implementación en la FPGA.

En el siguiente enlace se puede verificar la funcionalidad de la implementación realizada:

https://youtube.com/shorts/9Uha4O5wPcw?si=X9-zHAuQW8thO4DC

Ejercicio 3.4- Interfaz Serial Asíncrona

Este protocolo se ha utilizado por décadas para la comunicación entre sistemas. En este laboratorio estaremos desarrollando una interfaz UART (Universal Asynchronous Receiver/Transmitter). Con el bloque aquí desarrollado, la FPGA podrá comunicarse de forma bidireccional con otros sistemas de procesamiento (por ejemplo, un computador personal).

1. Para implementar la interfaz UART puede hacer uso de un código encontrado en algún repositorio cuya licencia sea libre. La integración y funcionamiento de dicho módulo corre por su cuenta.

2. Debe elaborar un set de pruebas para asegurar que dicho módulo UART funciona correctamente.

3. La interfaz UART a desarrollar debe ser capaz de manejar el protocolo serial bidireccional a una velocidad de 9600 baudios.

4. Desarrolle un diseño apropiado que permita enviar a una computadora personal un ”Hola mundo” al presionar un botón de la FPGA. Adicionalmente, el diseño debe mostrar en los LEDs datos recibidos desde dicha computadora (datos ASCII del teclado). En la figura 1 se ilustra el diagrama de bloques ideal para la prueba en físico.

5. Debe al menos elaborar una simulación de integración total de todos los bloques que haya elaborado: LEDs y UART.

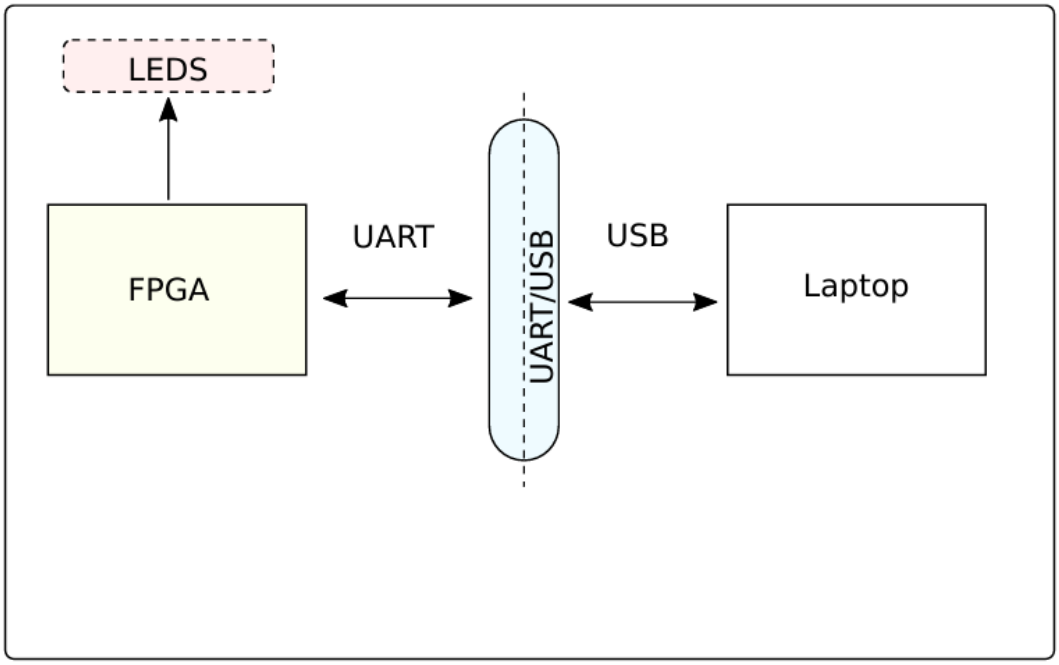

Figura 1: Diagrama conceptual de la interfaz UART conectado al computador y FPGA. LEDS muestran datos recibidos por el computador.

Diseño:

Primero se busco una UART hecha de licencia libre, que cumpliera con todos los requisitos del problema [2].

Entendimos la UART e hicimos un posible diagrama de bloques con la posible solución del problema:

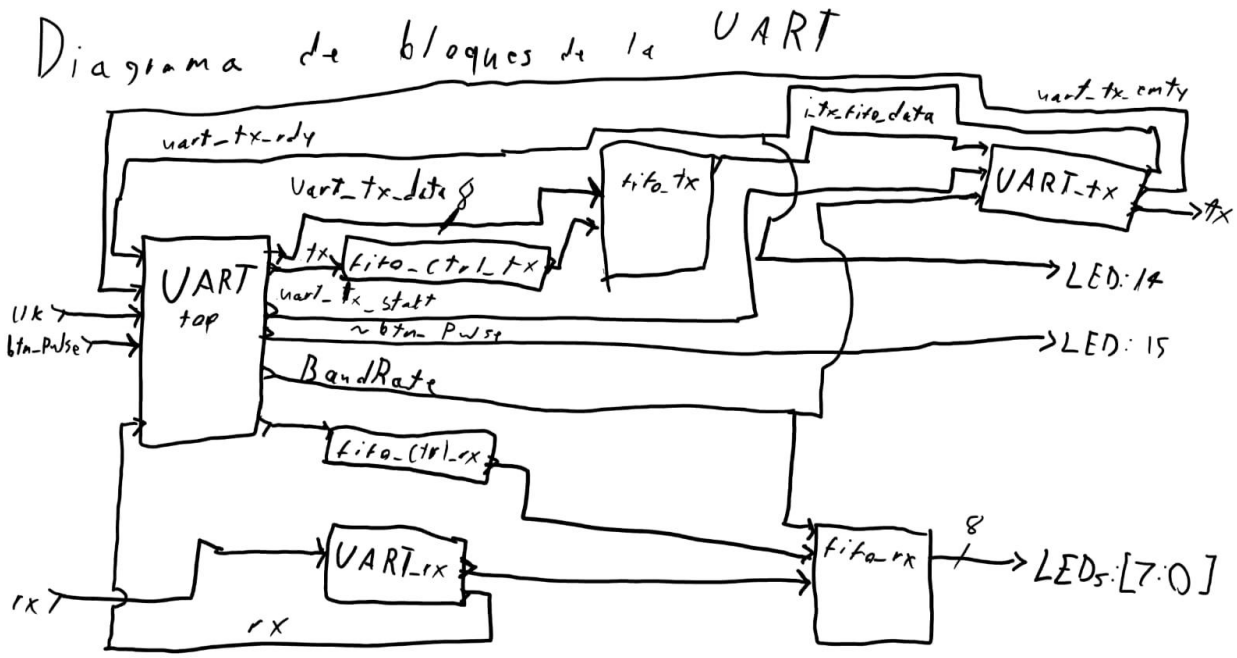

Fig.5 Diagrama de bloque UART, posible respuesta.

Además, que para ver que la UART funcionara se hizo un loopback, además de probar el propio testbench de la UART que venía en el repositorio, ambos funcionaron. Es importante anotar que la aplicación que se usó para poder comunicar la UART a la PC y viceversa fue Tera Teram.

Una vez que verificamos que la UART funcionaria se planteó una máquina de estado para insertarla dentro del Módulo total, que es la implementación del de la UART para el envío del “hola mundo” y el encendido de los leds al recibir una letra. La máquina de estado fue para la parte del “hola mundo. Como adicional se programó el LED 14 para que la FPGA nos dijera que la UART estaba lista.

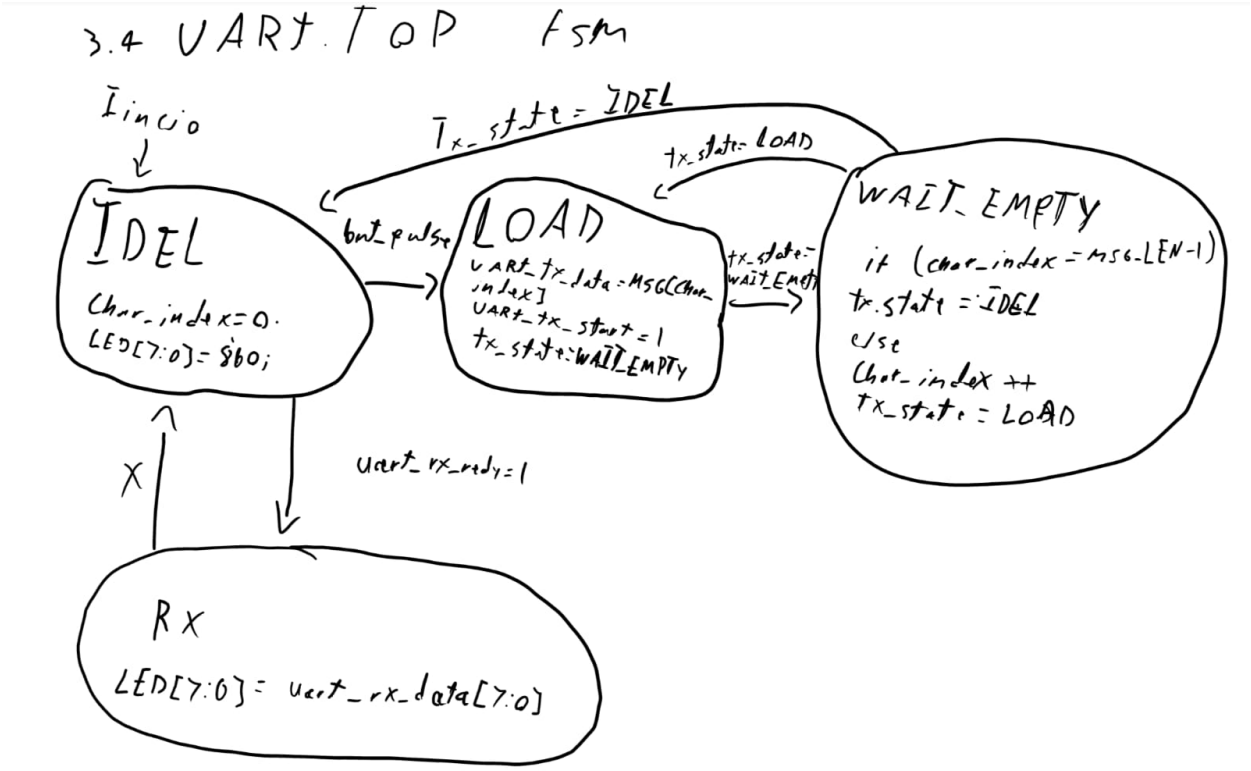

Fig.6 Maquina de estado para el “hola mundo”

Código:

https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/d9173cc679706c39cdeff0cb27f4bfaa175a95be/Laboratorio%201/problema%203.4

Constrains y Designed Sources

https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/39409e7e5de8d142ae922d6f70859434e66ab2de/Laboratorio%201/problema%203.4/Resolucion%20problema%203.4

Es necesario agregar los archivos de la Uart para que el Top_Todo funcione.

https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/39409e7e5de8d142ae922d6f70859434e66ab2de/Laboratorio%201/problema%203.4/UART%20usada

Simulation Sources

https://github.com/Oscar-Electronics/Taller_dise-o_digital/tree/39409e7e5de8d142ae922d6f70859434e66ab2de/Laboratorio%201/problema%203.4/UART%20usada/test

Resultados de simulación:

Uart_Tb como top

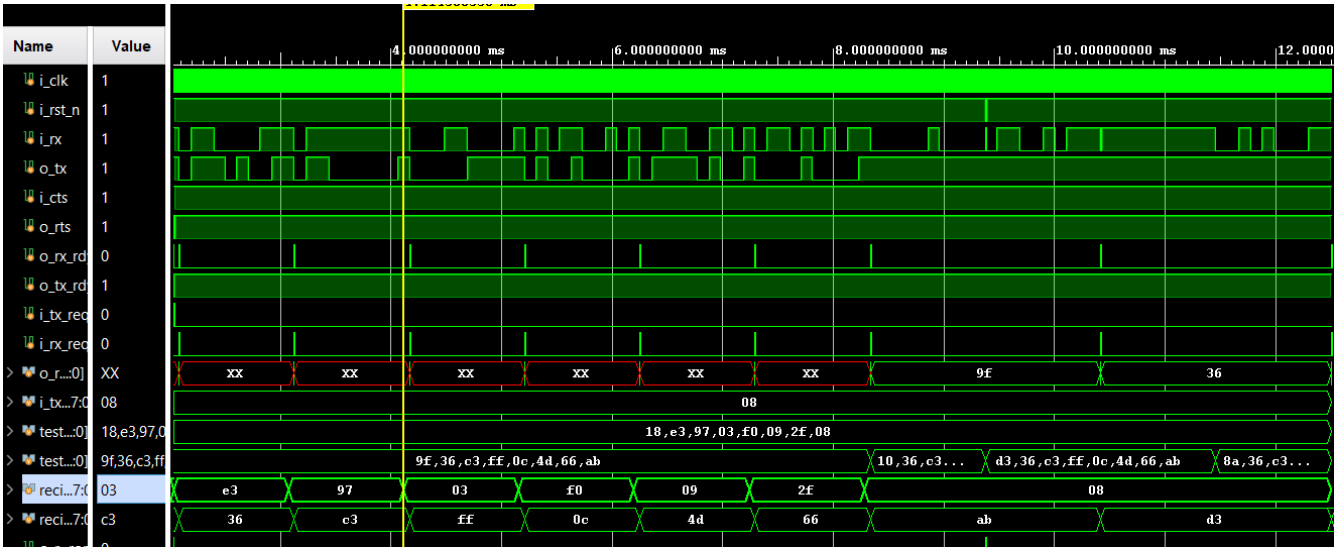

Fig 7. Oscilograma del funcionamiento de la UART

En el oscilograma de la figura 7 es posible ver cómo se envían una serie de datos y pasado un tiempo se reciben dichos datos demostrando como la interfaz de la UART si funciona de manera correcta, entonces podemos usarla sin preocupaciones en nuestro módulo de resolución del problema, Como es posible ver en el siguiente apartado, todo funciona.

Verificación de funcionamiento en la FPGA:

https://www.youtube.com/shorts/exAvfGnB9Y8

Referencias

[1] Digilent, «digilent-xdc/Nexys-4-DDR-Master.xdc at master · Digilent/digilent-xdc», *GitHub*. https://github.com/Digilent/digilent-xdc/blob/master/Nexys-4-DDR-Master.xdc

[2] Oskarwires, «GitHub - oskarwires/uart2: SystemVerilog UART transciever», GitHub. https://github.com/oskarwires/uart2.git